PROJECT PROPOSAL · CATEGORY 1: CLASSIC PAPER REPLICATION & STRESS TESTING

# Where Naive Bayes Breaks

*An adversarial stress test of a from-scratch text classifier: replicating Manning et al., Chapter 13, against classic and modern attacks*

Course topic: Beyond TF-IDF — An Experimental Study of Modern Document Retrieval | Group 2 | 21 September 2026

## 1. Group Details — Group No. 2

| Name | ID | Email |
|---|---|---|
| Nishant Pradhan | 2023A7PS0030P | f20230030@pilani.bits-pilani.ac.in |
| Shantnu Sarda | 2023A7PS0017P | f20230017@pilani.bits-pilani.ac.in |
| Arnav Sahoo | 2023A7PS0560P | f20230560@pilani.bits-pilani.ac.in |
| Tapan Chetanbhai Sanghani | 2024A7PS0562P | f20240562@pilani.bits-pilani.ac.in |

## 2. Project Title and Primary Objective

**Title.** Where Naive Bayes Breaks: Replication and Adversarial Stress-Testing of Multinomial and Bernoulli Naive Bayes Text Classification (Manning et al., Ch. 13 and §13.5) under Good-Word, Invisible-Text, LLM-Camouflage and Data-Poisoning Attacks.

**Primary objective.** Replicate the Naive Bayes (NB) text classifiers and the feature-selection methods of the course textbook [T, Ch. 13, §13.5] in NumPy/Pandas only — no wrapper libraries — build a fresh custom dataset, and deliberately attack the classifier to find its **breaking point**: how much adversarial effort (words added, characters injected, queries issued, training data poisoned) it takes to collapse Recall and F-measure, and which inexpensive defences restore them.

**Why spam filtering.** It is the setting where the textbook itself describes an economically motivated adversary [T, §19.2.2], and where NB is the historical workhorse — Bogofilter, CRM114, POPFile, SpamAssassin and SpamBayes are all NB-based [P5]. With spam as the positive class the metrics gain a concrete meaning: an evasion is a lost true positive (Recall drop), a poisoning success is a false positive (Precision drop).

**Research questions. RQ1** How many words does it take to break NB, for Multinomial vs Bernoulli, under passive / active / white-box good-word attacks, and how far is that from the analytic minimum? **RQ2** Does feature selection (method and size k) make NB more or less robust? **RQ3** Are encoding attacks (hidden text, zero-width characters, homoglyphs) cheaper than content attacks, and do simple sanitisers neutralise them? **RQ4** Can an LLM rewrite evade NB with zero model knowledge, and which tokens cause it? **RQ5** How much damage does a small fraction of poisoned training data do? **RQ6** (stretch) Does going "beyond TF-IDF" actually buy robustness?

**Mapping to the assignment's core tasks.** Replication in NumPy/Pandas with scikit-learn never imported by the pipeline (§6.1); custom data = **Fresh-100**, 100 documents from 14–20 Sep 2026 (§3); adversarial attacks A1–A8 including invisibly inserted good words (A5); and Precision, Recall and F-measure reported before and after every attack at every budget (§4).

## 3. Datasets under Consideration

| Dataset | Role and description | Link |
|---|---|---|
| Fresh-100 (custom, ours) | The assignment's custom data; out-of-time test set and attack target. 100 documents from 14–20 Sep 2026: 50 legitimate (≥40 news articles from ≥5 outlets and ≥4 topics, ≤10 public campus notices) and 50 unwanted (scam/phishing/promotional mail from members' own spam folders, public advertorial and coupon pages). Grown to ≥300 if time permits. | Collected by us from RSS feeds and spam folders. Only IDs, URLs and hashes are shared, never full text. |
| Enron-Spam (Metsis et al., 2006) | Main training and test set. Six sub-corpora of real Enron ham mixed with spam; file names carry order of arrival, so a temporal split is possible. Dataset paper: [P4]. | www2.aueb.gr/users/ion/data/enron-spam/ · mirror: huggingface.co/datasets/SetFit/enron_spam |
| SpamAssassin public corpus | External test set and the attacker's ham pool (~6,000 hand-labelled messages, ~31% spam). | spamassassin.apache.org/old/publiccorpus/ |
| Reuters-21578 (ModApte) | Replication sanity check for §13.5–13.6 and source of neutral news vocabulary; 9,603 train / 3,299 test documents — the benchmark used in the textbook's classification chapter. | archive.ics.uci.edu/dataset/137/reuters+21578+text+categorization+collection |
| Adversarial spam set (generated by us) | Artefact of A7 (LLM camouflage) and A8: attacked messages with defanged URLs and placeholder contact details, plus generation prompts and settings. | Released with our code. |

**Protocol.** Every Fresh-100 item stores source, timestamp and content hash, so the collection is auditable; two members label independently against written guidelines with a third adjudicating, and we report Cohen's κ. Enron-Spam is split temporally (oldest 70% train, newest 30% test) with hyper-parameters tuned on a dev slice only. Near-duplicates are removed by shingle hashing between any training set and any external test set, and the attacker's ham pool is always disjoint from the victim's training data. With 50 spam documents a 95% interval on Recall is roughly ±8 points at 90% recall, so Fresh-100 answers "does this hold on 2026 text?" while fine-grained comparisons run on the thousands of messages in Enron-Spam and SpamAssassin. A no-attack control on Fresh-100 is always reported so that temporal drift is never mistaken for adversarial damage. Personal data is scrubbed, every URL defanged, and nothing is clicked or re-sent.

## 4. Evaluation Criteria

Spam is the positive class: P = TP/(TP+FP), R = TP/(TP+FN), F1 = 2PR/(P+R) [T, §8.3]; macro- and micro-averaged F on Reuters sanity runs [T, §13.6]. Evasion lowers Recall, poisoning lowers Precision, so all three are always reported.

| Criterion | How it is measured | What we report |
|---|---|---|
| 1. Replication correctness | Textbook Example 13.1 (multinomial: 0.0003 vs 0.0001 → China) and Example 13.2 (Bernoulli: 0.005 vs 0.022 → not-China) reproduced exactly; log-posteriors match a reference implementation to 1e-9; Reuters-21578 reproduces the §13.5 trends (MI and $\chi^2$ reach similar peak accuracy, $\chi^2$ peaks at larger k). | All unit tests pass; trend plots included |
| 2. Clean performance | P, R, F1, accuracy and false-positive rate on Enron-Spam (temporal split), SpamAssassin and Fresh-100, per model and per feature-selection setting. | Consistent with published NB results [P4]; drift on Fresh-100 quantified |
| 3. Attack effectiveness | ASR (share of correctly flagged spam that evades after the attack), ΔRecall, ΔF1; breaking point b* = smallest budget at which F1 falls below half its clean value; robustness AUC under the normalised F1-vs-budget curve. | Before/after P, R, F1 tables; budget curves |
| 4. Attacker cost and stealth | Tokens added, % length increase, visible-character change rate, normalised edit distance, number of queries (A4). | Effort-vs-damage curves |
| 5. Defence value | F1 recovered at a fixed budget; clean-F1 penalty (accept ≤2 points); performance against an adaptive attacker that knows the defence. | Defence × attack matrix |
| 6. Statistical rigour | ≥5 seeds for randomised attacks; 95% bootstrap CIs (1,000 resamples); McNemar test for paired before/after comparisons. | CIs on every headline number; exact counts on Fresh-100 |
| 7. Reproducibility | Fixed seeds, versioned YAML configs, one-command re-run, dataset datasheet, pytest suite, code review. | Repository README + make reproduce |

## 5. Related Research: Approach, Findings and Use

**Textbook [T].** C. D. Manning, P. Raghavan and H. Schütze, *Introduction to Information Retrieval*, Cambridge University Press, 2008 — nlp.stanford.edu/IR-book/ · PDF. We use §2.2 (tokenisation and normalisation — also the gap that encoding attacks exploit), §6.2 (tf-idf, the "beyond TF-IDF" framing and the S1 baseline), §8.3 (P/R/F definitions), §13.1–13.4 (the algorithms we replicate; Examples 13.1 and 13.2 as exact unit tests; add-one smoothing, log-space scoring), §13.5 (the three feature selectors and the trends we reproduce — the axis of RQ2), §13.6 (macro/micro-averaged F), §14.2 (Rocchio, for S1), §19.2.2 (spam and adversarial IR — the adversary's motive) and §21.2 (PageRank, for stretch S3).

| Paper (title, venue, link) | Approach and key findings | Use in our project |
|---|---|---|
| **P1 A Comparison of Event Models for Naive Bayes Text Classification** — McCallum & Nigam, AAAI-98 Workshop. cs.tufts.edu/~nr/cs257/archive/andrew-mccallum/event-models.pdf | Distinguishes the multivariate Bernoulli (word presence) and multinomial (word counts) event models and compares them on five corpora: Bernoulli is competitive at small vocabularies, the multinomial generally wins as vocabulary grows. | Defines the two variants we implement; motivates studying how vocabulary size interacts with robustness. Schneider (EACL 2003, aclanthology.org/E03-1059.pdf) repeats the comparison for spam and finds the multinomial less class-biased — our expected clean-data ordering. |
| **P2 Tackling the Poor Assumptions of Naive Bayes Text Classifiers** — Rennie et al., ICML 2003. web.stanford.edu/class/cs276/handouts/rennie.icml03.pdf | Treats NB as a linear classifier and proposes heuristic corrections for its systemic weaknesses and for the fact that text is not truly multinomial; corrected NB becomes SVM-competitive. Cited in [T, Ch. 13] further reading. | Confirms that the weights $w_t$ are the right object to analyse; term-frequency transforms become defence D2. |
| **P3 A Comparative Study on Feature Selection in Text Categorization** — Yang & Pedersen, ICML 1997. mlanthology.org/icml/1997/yang1997icml-comparative/ | Compares document frequency, information gain, mutual information, $\chi^2$ and term strength: IG and $\chi^2$ work best, up to 98% of unique terms can be removed without hurting accuracy, and MI performs poorly because it favours rare terms. Cited in [T, §13.5]. | Justifies our selector grid. They asked what selection does to accuracy; we ask what it does to robustness. |
| **P4 Spam Filtering with Naive Bayes: Which Naive Bayes?** — Metsis et al., CEAS 2006. core.ac.uk/works/16760537 | Compares five NB variants on six new Enron-based datasets that preserve message order and realistic spam:ham ratios, and releases the datasets. | Source of our main corpus and of the temporal-split protocol. |
| **P5 Good Word Attacks on Statistical Spam Filters** — Lowd & Meek, CEAS 2005. personal.utdallas.edu/~mxk055100/courses/dmsec_files/125.pdf | Defines the good-word attack and evaluates passive (no feedback) and active (with filter feedback) variants against NB and maximum-entropy filters: a passive attacker gets about half of blocked spam through, random or common-word padding does little against NB while "hammy" words work, and active attacks are very effective. | Blueprint for attacks A1–A4 and for our passive / active / white-box threat models. |
| **P6 Exploiting Machine Learning to Subvert Your Spam Filter** — Nelson et al., USENIX LEET 2008. usenix.org/conference/leet-08/exploiting-machine-learning-subvert-your-spam-filter | Training-time contamination attacks on SpamBayes: a dictionary attack renders the filter useless with control of only ~1% of training messages, and focused attacks block chosen messages. Proposes RONI (reject on negative impact) and dynamic thresholds. | Basis for poisoning attack A8 and the RONI-lite defence D5. |
| **P7 Bad Characters: Imperceptible NLP Attacks** — Boucher et al., IEEE S&P 2022. arxiv.org/abs/2106.09898 | Black-box attacks using invisible characters, homoglyphs, reorderings and deletions that leave text visually unchanged; one injection significantly degrades models and three break most of them, including commercial systems. Concludes that input sanitisation is required. | Basis for A6 and defence D1 — the "invisible" half of the assignment's twist. |

| Paper (title, venue, link) | Approach and key findings | Use in our project |
|---|---|---|
| **P8 Is BERT Really Robust? (TextFooler)** — Jin et al., AAAI 2020. arxiv.org/abs/1907.11932 | Black-box attack that ranks word importance and swaps in semantically similar words, fooling BERT, CNN and RNN classifiers while preserving meaning, at cost linear in text length. | Template for a synonym-substitution variant of A4 and for the neural comparison in S1. |
| **P9 Investigating the Effectiveness of Bayesian Spam Filters in Detecting LLM-modified Spam Mails** — Josten & Weis, arXiv 2024. arxiv.org/abs/2408.14293 | Rewrites spam with GPT-3.5 Turbo and tests SpamAssassin: up to 73.7% of LLM-modified spam is misclassified as legitimate, versus at most 0.4% for a simple dictionary-replacement attack, at roughly 0.17 cents per e-mail. | Motivates A7. We repeat the idea against a transparent from-scratch NB so we can explain why it evades. |
| **P10 Adversarial Web Search** — Castillo & Davison, FnTIR 4(5), 2011. ccs.neu.edu/home/vip/teach/IRcourse/IR_surveys/adversarial-web-search.pdf | Survey of adversarial IR: content spam, cloaking, link bombs and link farms, and the corresponding detection methods — the research area [T, §19.2.2] points to. | Threat-model vocabulary; background for the PageRank spam-farm stretch (S3). |
| **P11 Nightmare at Test Time: Robust Learning by Feature Deletion** — Globerson & Roweis, ICML 2006. mlanthology.org/icml/2006/globerson2006icml-nightmare/ | Minimax formulation that avoids putting too much weight on any single feature so the classifier tolerates feature deletion; solved by quadratic programming and illustrated on spam filtering. | Inspiration for weight clipping (D3) and the certified token-edit radius (S2). |
| **P12 A Multiple Instance Learning Strategy for Combating Good Word Attacks on Spam Filters** — Jorgensen et al., JMLR 9, 2008. jmlr.org/papers/v9/jorgensen08a.html | Splits each e-mail into segments and applies multiple-instance logistic regression, so appended good words cannot mask the spam portion. | Inspiration for segment-wise NB scoring (D4). |
| **P13 Adversarial Feature Selection against Evasion Attacks** — Zhang et al., IEEE Trans. Cybernetics 2016. arxiv.org/abs/2005.12154 | Shows that feature selection can worsen classifier security against evasion, and proposes an adversary-aware wrapper-based selector, evaluated on spam and malware. | Direct motivation for RQ2. Bao et al. (arXiv 1808.05705) report the opposite — embedded selection as a mitigation — so our NB results will show which view holds for text. |
| **P14 Is Feature Selection Secure against Training Data Poisoning?** — Xiao et al., ICML 2015. proceedings.mlr.press/v37/xiao15.html | Shows LASSO, ridge and elastic-net selection can be compromised; on malware data under 5% poisoned samples reduce LASSO to almost random feature choices. | Motivates testing MI, $\chi^2$ and frequency selection under poisoning (A8), not only under evasion. Miller et al. (arXiv 1811.00121) add that one density per class is a weak model and defend with an NB mixture — a candidate extension of D5. |
| **P15 Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks** — Reimers & Gurevych, EMNLP 2019. arxiv.org/abs/1908.10084 | Fine-tunes BERT in a siamese setup to produce dense sentence embeddings usable with cosine similarity or a simple classifier. | Modern dense-representation baseline for the S1 comparison (all-MiniLM-L6-v2, ~22M parameters). |

**What the literature leaves open.** (i) Good-word attacks are well documented [P5], but we found no study pairing an *analytic* minimum-effort bound for NB with empirical passive and active curves for both NB variants. (ii) Feature-selection security studies target LASSO, SVM and malware data and disagree with each other [P12 vs Bao et al.]; we did not find the textbook's own MI, $\chi^2$ and frequency selectors evaluated against good-word, hidden-text and poisoning attacks on NB — we will confirm this gap in Week 1 and soften the claim if prior work exists. (iii) [P9] measures LLM camouflage against SpamAssassin, a black box; against a from-scratch NB we can attribute an evasion to specific tokens and margins. (iv) [P7] targets modern NLP pipelines, while a bag-of-words model is even more exposed, since one injected character changes a token's identity.

## 6. Proposed Approach and Architecture

**Pipeline.** data (Enron-Spam / SpamAssassin / Reuters / Fresh-100) → tokenizer with optional sanitiser D1 → victim models (Multinomial NB, Bernoulli NB, each × selector × k) → attack lab (A1–A8) → evaluation harness (P/R/F1, ASR, breaking point, CIs) → defences D1–D6 → adaptive-attack loop.

**Design principles.** Every attack is tied to an explicit attacker capability and a budget unit, so results are comparable across attacks. Because NB is linear in log-space we compute the minimum attacker effort analytically before any trial and error. A defence counts as a success only if it survives an attacker who knows it. And controls are everywhere: a random-padding control (A1), a no-attack drift control on Fresh-100, and a disjoint attacker pool.

### 6.1 Replicating Naive Bayes and feature selection [T, Ch. 13]

- Multinomial NB [T, §13.2]: priors $\hat{P}(c) = N_c/N$; $\hat{P}(t|c) = (T_{ct} + 1)/(\Sigma_{t'} T_{ct'} + |V|)$ with add-one smoothing; scoring entirely in log-space to avoid underflow.
- Bernoulli NB [T, §13.3]: $\hat{P}(t|c) = (N_{ct} + 1)/(N_c + 2)$; every vocabulary term contributes whether present or absent. Efficient scoring precomputes $\Sigma \log(1 - p_t)$ and adds one correction per present term.
- Feature selection [T, §13.5]: expected mutual information, $\chi^2$ on the 2×2 term–class contingency table, and document-frequency ranking; for multi-class Reuters we select per class and take the union, as the textbook prescribes.
- No wrapper libraries. Documents become integer token-id arrays; class term counts from np.bincount, document frequencies from unique (document, term) pairs, scores from np.add.reduceat over concatenated id arrays. This runs comfortably on tens of thousands of messages with no sparse-matrix library.
- Verification. Examples 13.1 and 13.2 as exact unit tests; property tests (probabilities sum to one, invariance to document order, correct handling of unseen terms); an isolated cross-check against a reference implementation.

### 6.2 Threat models and attack suite

**T1 passive oblivious** (knows only that the filter is a statistical text classifier; negative control) → A1. **T2 passive gray-box** (knows the model family and has a public ham corpus disjoint from training data, no feedback) → A2, A5, A7. **T3 active black-box** (may query the filter for a

label, up to Q queries per message) → A4. **T4 white-box** (knows every weight $w_t$; gives the oracle lower bound) → A3, A5, A6. **T5 training contamination** (controls x% of spam-labelled training messages, the assumption of [P6]) → A8. In all cases the spam payload — the pitch and a placeholder URL — must stay intact and readable; hidden content is allowed only in A5.

| ID | Attack | Threat | Mechanism | Budget unit |
|---|---|---|---|---|
| A1 | Random / common-word padding (control) | T1 | Append b random vocabulary words, or the b most frequent English words. Expected to be weak against NB [P5]. | Tokens |
| A2 | Passive good-word | T2 | Append the top-b words ranked by an estimated ham-vs-spam log-ratio computed from the attacker's own ham pool and known spam [P5]. | Tokens |
| A3 | White-box optimal good-word | T4 | Greedy on the true weights: (i) distinct-word variant, (ii) repetition variant for multinomial NB. Gives the analytic bound (§6.3). | Tokens |
| A4 | Active good-word | T3 | Test candidate words through label queries, keep those that lower the spam score, stop at the flip [P5]. Variant with synonym substitution in the spirit of [P8]. | Tokens, queries |
| A5 | Hidden-text injection | T2 / T4 | Good words placed in HTML a human cannot see (display:none, zero font size, white-on-white). Evaluated under a naive extractor and a render-aware one. | Hidden tokens (0 visible characters) |
| A6 | Unicode tricks | T4 | Zero-width characters, homoglyphs and leetspeak on the m most spam-indicative tokens so they become unseen tokens [P7]. | Characters edited |
| A7 | LLM camouflage | T2 | An open-weight or free-tier LLM rewrites the spam in a legitimate register (paraphrase only, then paraphrase plus benign padding), preserving the pitch, with no knowledge of the victim [P9]. | Rewrite; tokens changed |
| A8 | Data poisoning | T5 | Inject $x \in \{0.5, 1, 2, 5\}$% spam-labelled messages stuffed with ham vocabulary (dictionary attack) [P6]. | % of training set |

## 6.3 Analytic breaking point

For multinomial NB the score is linear in term counts — $S(d) = \log \hat{P}(\text{spam})/\hat{P}(\text{ham}) + \Sigma_t\, tf_t \cdot w_t$, with $w_t = \log \hat{P}(t|\text{spam})/\hat{P}(t|\text{ham})$ — and a message is flagged when $S(d) > 0$. Appending one token changes the score by $w_t$, so a hammy token ($w_t < 0$) lowers it by $|w_t|$. The minimum number of *distinct* hammy words needed to flip a spam message of margin $S(d)$ is therefore $b^*(d) = \min \{ b : \Sigma_{i=1..b} |w_{(i)}| \geq S(d) \}$, where $w_{(1)}, w_{(2)}, \ldots$ are the most negative weights in descending magnitude.

- With repetition (multinomial NB) a single hammy word repeated $\lceil S(d)/|w_{min}| \rceil$ times flips any spam message: there is no ceiling on padding-based evasion, only a price in tokens — and we measure that price.
- Bernoulli NB counts presence only: a word contributes $\delta_t = \log(p_t^{spam}/p_t^{ham}) - \log((1-p_t^{spam})/(1-p_t^{ham}))$ once, so repetition is useless and the attacker is limited by the number of distinct hammy words in the vocabulary.
- Use. The white-box $b^*(d)$ is the oracle lower bound; the gap between it and the passive (A2) and active (A4) curves quantifies exactly what attacker knowledge is worth.

**Hypotheses (to be tested, not assumed). H1** Padding is cheapest against multinomial NB because of repetition; Bernoulli NB is far less padding-sensitive but exposed to distinct-word attacks. **H2** Feature selection has two opposing effects — it shrinks the pool of hammy words and makes out-of-vocabulary padding useless, but the surviving words carry the largest $|w_t|$; we expect the first effect to dominate for distinct-word attackers and the second for repetition attackers, and since $\chi^2$ keeps rarer terms than MI [T, §13.5] we expect $\chi^2$ to be more exposed to poisoning. **H3** Encoding attacks (A5, A6) are cheapest in visible effort, and D1 neutralises most of them at negligible clean cost. **H4** LLM camouflage evades NB without model knowledge because NB keys on lexical spam markers a rewrite removes; segment scoring (D4) helps against padding but much less against a full rewrite. **H5** Temporal drift alone reduces Recall on Fresh-100, which is why the no-attack control is mandatory.

## 6.4 Experiment grid and defences

**Grid.** model {Multinomial, Bernoulli} × selector {none, frequency, MI, $\chi^2$} × k ∈ {25, 50, 100, 300, 1,000, 3,000, all} × attack {A1–A8} × budget {0, 5, 10, 20, 50, 100, 200, 500 tokens or the attack's own unit}. Scoring NB is one vectorised sweep, so several thousand cells run in minutes; randomised attacks repeat over 5 seeds and the only expensive step is LLM generation (~300 messages × 2–3 prompts). Outputs are F1-vs-budget curves per selector and k, breaking-point heat-maps (k × budget), and per-document diagnostics (margin, top contributing tokens before and after).

| ID | Defence | Counters | Mechanism | Inspired by |
|---|---|---|---|---|
| D1 | Canonicalise + render-aware extraction | A5, A6 | Unicode NFKC, strip zero-width and control characters, map confusables to base Latin, drop HTML text that is not visible. | [P7] |
| D2 | Term-count cap / binarisation | A2, A3 | Cap tf at c (c = 1 is Bernoulli-like), so repetition stops paying. | [T, §13.3]; [P2] |
| D3 | Weight clipping | All token-level | Clip $|w_t| \leq \tau$ so no single word dominates; yields a certified radius (S2). | [P11] |
| D4 | Segment-wise scoring | A2–A4 | Split a message into segments, score each, flag if any (or the max) segment is spam. | [P12] |
| D5 | RONI-lite filter | A8 | Reject a candidate training message if adding it lowers held-out F1 by more than a threshold. | [P6] |
| D6 | Attack-aware retraining | All | Retrain on attacked samples from a previous round, then face the next-round attacker (arms-race curves). | [P5] |

***Adaptive attackers:*** *against D4, interleave good words sentence by sentence; against D3, pad with more tokens; against D5, spread poison thinly to stay under the RONI threshold; against D1, look for character classes missing from the confusables map.*

**Why weight clipping is more than a heuristic.** If every weight is clipped to $[-\tau, \tau]$ at inference, one token insertion or deletion changes the score by at most $\tau$, so a message of margin $S(d)$ cannot be flipped by fewer than $r(d) = \lceil S(d)/\tau \rceil$ token edits — a certified lower bound that stretch objective S2 turns into certified-accuracy curves.

**Why this approach.** NB is the textbook's canonical text classifier, still used in real filters [P5], and its log-linear form lets us *explain* attacks exactly rather than treating them as trial and error. Spam gives the adversary a motive and the metrics a meaning [T, §19.2.2]. From-scratch

code is a feature, not a constraint: we can instrument weights, margins and per-token contributions, which a black-box library call makes impossible. And the four attack layers map to four distinct failure causes — content (statistics), encoding (tokenizer), generation (lexical markers) and training data (estimation) — each with a matching cheap defence.

## 7. Key Innovations

The individual attacks come from the literature. What is new is the combination, the rigour, and the questions asked of a classic model.

- **Breaking point as a budget-normalised, analytically anchored number.** The white-box oracle b*(d) is compared against passive and active empirical curves; the gap measures the value of attacker knowledge, and the curves are comparable across attacks and models.
- **Feature selection treated as a security variable.** To our knowledge (§5) the first systematic look at MI, $\chi^2$ and frequency selection × Multinomial/Bernoulli NB × four attack families — on a question the literature disagrees about [P12 vs Bao et al.].
- **One threat-model-indexed harness for four attack layers.** Content, encoding, generation and training-time attacks evaluated on the same data, splits, metrics and budget conventions.
- **Attack → defence → adaptive-attack loop.** Cheap, explainable defences (D1–D6) are always retested against an attacker who knows them, plus a certified token-edit radius for clipped NB (S2).
- **Fresh 2026 data with an explicit drift control.** Separates "old model on new text" from "model under attack" — a confound many robustness studies leave unaddressed.
- **Explainable failures.** Per-document margins and top contributing tokens before and after each attack show why NB broke, which is only possible because we implement it ourselves.

## 8. Stretch Objectives

Attempted only after the "Must" tier of §9 is complete.

| ID | Objective | What we would do and why it is interesting | Priority |
|---|---|---|---|
| S1 | Beyond TF-IDF: is "modern" more robust? | Run the full attack suite against (i) TF-IDF + Rocchio [T, §6.2, §14.2] written from scratch and (ii) MiniLM embeddings [P15] + logistic regression. Embedding models truncate long inputs (default 256 word-pieces), so where good words are inserted (prepend vs append) should matter — a testable prediction. Directly answers RQ6 and ties the project to the course topic. | High |
| S2 | Certified token-edit radius for clipped NB | With weights clipped to $[-\tau, \tau]$ (D3), report certified accuracy vs radius against the empirical breaking point, and the clean-F1 cost of small $\tau$. | High |
| S3 | PageRank spam-farm mini-study [T, Ch. 21, §19.2.2; P10] | Build a small hyperlink graph from the collected articles and their neighbours, implement PageRank by power iteration in NumPy, add a 10-page spam farm pointing at a target page, measure rank inflation, and test a trust-seeded (biased) PageRank defence; topic-specific PageRank if time allows. | Medium |
| S4 | Arms-race dynamics | Three rounds of attack → retrain (D6) → adaptive attack. Does the defender catch up, or does the attacker always stay ahead? | Medium |
| S5 | Stealth study | Small human rating study (≥8 raters) on readability and suspiciousness of attacked messages, to calibrate the visible-perturbation metric. | Low |
| S6 | Release AdvNB-Bench | Code, defanged attacked messages, generation prompts and datasheet as a reusable benchmark for classic-model robustness. | Low |

## 9. Key Milestones and Tentative Timeline

An eight-week schedule starting the week of this proposal; dates are provisional and will be aligned to the official course deadlines.

| | Week ends | Deliverable and exit criterion |
|---|---|---|
| M1 | W1 · 27 Sep | Repository and environment ready; Fresh-100 collection window closed; Multinomial and Bernoulli NB pass textbook Examples 13.1 and 13.2 exactly. |
| M2 | W2 · 4 Oct | Feature selection (frequency, MI, $\chi^2$) done; Reuters-21578 sanity check reproduces the §13.5 trends; Fresh-100 labelled with $\kappa$ reported; clean P/R/F1 baselines on all three corpora. |
| M3 | W3 · 11 Oct | Attack harness plus A1–A3; first breaking-point curves; analytic-vs-empirical comparison for both NB variants. |
| M4 | W4 · 18 Oct | A4–A6 and the full feature-selection × attack grid for both NB variants. Mid-project checkpoint / progress demo. |
| M5 | W5 · 25 Oct | A7 (LLM camouflage) and A8 (poisoning) complete; defences D1–D5 implemented. |
| M6 | W6 · 1 Nov | Adaptive attacks and D6; stretch S1 and S2 under way; experiment configurations frozen. |
| M7 | W7 · 8 Nov | All multi-seed runs finished; confidence intervals, figures and error analysis; results freeze. |
| M8 | W8 · 15 Nov | Final report, slides, demo, clean repository and README; buffer week. |

**Scope ladder (protects the core deliverable). Must:** NB replication (both variants) and feature selection; Fresh-100; clean baselines; A1, A2, A3, A5, A6; analytic breaking point; before/after P, R, F1 with CIs; D1 and D2. **Should:** A4, A7, A8; the full selector × attack grid; D3–D5; adaptive attackers. **Could:** S1–S6 in the priority order of §8.

## 10. Other Relevant Information

| Risk | Mitigation |
|---|---|
| 100 documents is too few, or labels are noisy | Two annotators plus adjudication; grow Fresh to ≥300; use Enron-Spam and SpamAssassin for statistical power; report exact counts and CIs. |
| Scope creep (4 attack layers × 2 models × selector grid) | Must/Should/Could tiers (§9); weekly milestone review; vectorised code keeps the grid cheap. |
| Domain shift weakens the clean baseline on Fresh-100 | Drift control, and as fallback an in-domain split of the enlarged Fresh set; report both. |

| Risk | Mitigation |
|---|---|
| LLM access, cost, or refusals when asked to rewrite spam | Open-weight instruction-tuned model locally or on free GPU hours, or a free-tier API; fallback is a rule-based paraphraser with templates. |
| Numerical bugs (underflow, unseen terms, smoothing) | Log-space throughout; add-one smoothing; unit tests against the textbook examples plus property tests. |
| Data licensing and privacy | Share only IDs, URLs and hashes; scrub personal data; academic use only; follow the SpamAssassin corpus warning never to feed the messages into a live mail system. |

**Ethics.** Attacks are run only offline against our own classifier — nothing is sent to real mail systems or users. All URLs and contact details in generated or collected spam are defanged or replaced by placeholders, so the released artefact contains no working scam infrastructure. News text is used for research only and not redistributed; scraping is limited to public RSS feeds and respects site terms. The defences are part of the deliverable, so the work supports defenders as much as attackers.

**Assumptions and scope.** Binary task (spam vs legitimate), English-only, text-only — no images or attachments; spam is the positive class. ASR is computed on the "attackable set" (spam the clean model flags correctly), while P, R and F1 are computed on the full test set. We make no numeric claims about results in this proposal: H1–H5 are hypotheses, and the final report will state which were supported.

**Tools and reproducibility.** Python 3.11 with NumPy, Pandas and Matplotlib, plus standard-library text handling (re, unicodedata, html.parser). scikit-learn is never imported by the pipeline; it appears, if at all, only in an isolated test that cross-checks our log-posteriors, and we will drop it if the instructors prefer. Stretch-only dependencies are a sentence-embedding model for S1 (the logistic regression on top is written in NumPy) and an LLM for A7. Practice: fixed seeds, YAML experiment configs, a pytest suite, Git with reviews, and a single command that regenerates every table and figure.

**Deliverables.** Source repository with README, tests and one-command reproduction; the Fresh-100 datasheet (provenance, guidelines, κ) and the defanged adversarial set with prompts; a final report containing replication results, before/after Precision, Recall and F-measure tables, breaking-point curves, feature-selection heat-maps and a defence matrix; and slides with a short live demo in which a spam message is flipped to ham by a chosen attack, with the per-token explanation.

**Reference [T]:** C. D. Manning, P. Raghavan and H. Schütze, *Introduction to Information Retrieval*, Cambridge University Press, 2008 — https://nlp.stanford.edu/IR-book/. Papers P1–P15 are listed with titles and links in §5.